

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
dataset_path = "/content/drive/MyDrive/Alzheimer dataset"
```

```
import os

print(os.listdir(dataset_path))

['test', 'train']
```

```
print(os.listdir(dataset_path + "/train"))

['Final CN JPEG-20250126T183256Z-001.zip', '.DS_Store', 'Final MCI JPEG-20250126T183259Z-001.zip', 'Final LMCI JPEG-20250126T183259Z-001.zip', 'Final AD JPEG-20250126T183259Z-001.zip']
```

```
import os
import shutil
import random

base_dataset = "/content/drive/MyDrive/Alzheimer dataset"
original_train = os.path.join(base_dataset, "train")
working_dir = "/content/alzheimers_working"
train_dir = os.path.join(working_dir, "train")
val_dir = os.path.join(working_dir, "val")
test_dir = os.path.join(working_dir, "test")

classes = [
    "Final AD JPEG",
    "Final CN JPEG",
    "Final EMCI JPEG",
    "Final LMCI JPEG",
    "Final MCI JPEG"
]

# fresh working folders
if os.path.exists(working_dir):
    shutil.rmtree(working_dir)

os.makedirs(train_dir, exist_ok=True)
os.makedirs(val_dir, exist_ok=True)
os.makedirs(test_dir, exist_ok=True)

# split train into new train/val
random.seed(42)

for cls in classes:
    src_class_dir = os.path.join(original_train, cls)
    images = [img for img in os.listdir(src_class_dir)
               if img.lower().endswith((".jpg", ".jpeg", ".png"))]

    random.shuffle(images)
    split_idx = int(0.8 * len(images))

    train_images = images[:split_idx]
    val_images = images[split_idx:]

    os.makedirs(os.path.join(train_dir, cls), exist_ok=True)
    os.makedirs(os.path.join(val_dir, cls), exist_ok=True)

    for img in train_images:
        shutil.copy2(os.path.join(src_class_dir, img),
                     os.path.join(train_dir, cls, img))

    for img in val_images:
        shutil.copy2(os.path.join(src_class_dir, img),
                     os.path.join(val_dir, cls, img))

# copy test set as-is
original_test = os.path.join(base_dataset, "test")
for cls in classes:
```

```

src_class_dir = os.path.join(original_test, cls)
os.makedirs(os.path.join(test_dir, cls), exist_ok=True)

images = [img for img in os.listdir(src_class_dir)
           if img.lower().endswith((".jpg", ".jpeg", ".png"))]

for img in images:
    shutil.copy2(os.path.join(src_class_dir, img),
                 os.path.join(test_dir, cls, img))

print("Working dataset created.")
print("Train classes:", os.listdir(train_dir))
print("Val classes:", os.listdir(val_dir))
print("Test classes:", os.listdir(test_dir))

```

```

Working dataset created.
Train classes: ['Final CN JPEG', 'Final LMCI JPEG', 'Final EMCI JPEG', 'Final MCI JPEG', 'Final AD JPEG']
Val classes: ['Final CN JPEG', 'Final LMCI JPEG', 'Final EMCI JPEG', 'Final MCI JPEG', 'Final AD JPEG']
Test classes: ['Final CN JPEG', 'Final LMCI JPEG', 'Final EMCI JPEG', 'Final MCI JPEG', 'Final AD JPEG']

```

```

for split_name, split_path in [("train", train_dir), ("val", val_dir), ("test", test_dir)]:
    print(f"\n{split_name.upper()}")
    for cls in classes:
        count = len([img for img in os.listdir(os.path.join(split_path, cls))
                     if img.lower().endswith((".jpg", ".jpeg", ".png"))])
        print(cls, count)

```

```

TRAIN
Final AD JPEG 116
Final CN JPEG 394
Final EMCI JPEG 163
Final LMCI JPEG 48
Final MCI JPEG 158

```

```

VAL
Final AD JPEG 29
Final CN JPEG 99
Final EMCI JPEG 41
Final LMCI JPEG 13
Final MCI JPEG 40

```

```

TEST
Final AD JPEG 26
Final CN JPEG 87
Final EMCI JPEG 36
Final LMCI JPEG 11
Final MCI JPEG 35

```

```

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# define constants
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

# baseline preprocessing
baseline_datagen = ImageDataGenerator(rescale=1./255)

train_baseline = baseline_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True
)

val_baseline = baseline_datagen.flow_from_directory(
    val_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

test_generator = baseline_datagen.flow_from_directory(
    test_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,

```

```

class_mode='categorical',
shuffle=False
)

```

Found 879 images belonging to 5 classes.
Found 222 images belonging to 5 classes.
Found 195 images belonging to 5 classes.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization

```

```

cnn_model_1 = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224, 224, 3)),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(128, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax')
])

```

```

cnn_model_1.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
cnn_model_1.summary()
```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 222, 222, 32)	896
batch_normalization_3 (BatchNormalization)	(None, 222, 222, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_4 (Conv2D)	(None, 109, 109, 64)	18,496
batch_normalization_4 (BatchNormalization)	(None, 109, 109, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_5 (Conv2D)	(None, 52, 52, 128)	73,856
batch_normalization_5 (BatchNormalization)	(None, 52, 52, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten_1 (Flatten)	(None, 86528)	0
dense_4 (Dense)	(None, 128)	11,075,712
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 5)	645

Total params: 11,170,501 (42.61 MB)
Trainable params: 11,170,053 (42.61 MB)

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

```

early_stop = EarlyStopping(
    monitor='val_loss',

```

```

        patience=3,
        restore_best_weights=True
    )

    checkpoint = ModelCheckpoint(
        "/content/drive/MyDrive/best_cnn_model_1.keras",
        monitor='val_loss',
        save_best_only=True
    )

    history_1 = cnn_model_1.fit(
        train_baseline,
        validation_data=val_baseline,
        epochs=15,
        callbacks=[early_stop, checkpoint]
    )

```

```

Epoch 1/15
28/28 ----- 160s 6s/step - accuracy: 0.3663 - loss: 10.7776 - val_accuracy: 0.0586 - val_loss: 8.0084
Epoch 2/15
28/28 ----- 159s 6s/step - accuracy: 0.4437 - loss: 1.6973 - val_accuracy: 0.0856 - val_loss: 12.0551
Epoch 3/15
28/28 ----- 155s 5s/step - accuracy: 0.4437 - loss: 1.9936 - val_accuracy: 0.1036 - val_loss: 15.7077
Epoch 4/15
28/28 ----- 156s 6s/step - accuracy: 0.4460 - loss: 1.7730 - val_accuracy: 0.0811 - val_loss: 21.2060

```

```

norm_datagen = ImageDataGenerator(
    rescale=1./255,
    samplewise_center=True,
    samplewise_std_normalization=True
)

train_norm = norm_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True
)

val_norm = norm_datagen.flow_from_directory(
    val_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

```

```

Found 879 images belonging to 5 classes.
Found 222 images belonging to 5 classes.

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization

cnn_model_1_norm = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224, 224, 3)),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),

    Conv2D(128, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax')
])

cnn_model_1_norm.compile(
    optimizer='adam',
    loss='categorical_crossentropy',

```

```
metrics=['accuracy']
)
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

```
history_1_norm = cnn_model_1_norm.fit(
    train_norm,
    validation_data=val_norm,
    epochs=15,
    callbacks=[early_stop]
)
```

```
Epoch 1/15
28/28 ————— 160s 6s/step - accuracy: 0.3424 - loss: 14.8181 - val_accuracy: 0.4550 - val_loss: 1.6473
Epoch 2/15
28/28 ————— 156s 6s/step - accuracy: 0.4551 - loss: 1.6026 - val_accuracy: 0.2432 - val_loss: 1.6511
Epoch 3/15
28/28 ————— 157s 6s/step - accuracy: 0.4528 - loss: 1.6274 - val_accuracy: 0.0676 - val_loss: 2.4070
Epoch 4/15
28/28 ————— 156s 6s/step - accuracy: 0.4482 - loss: 1.5707 - val_accuracy: 0.3784 - val_loss: 1.5984
Epoch 5/15
28/28 ————— 157s 6s/step - accuracy: 0.4482 - loss: 1.5609 - val_accuracy: 0.4324 - val_loss: 1.5829
Epoch 6/15
28/28 ————— 157s 6s/step - accuracy: 0.4494 - loss: 1.5494 - val_accuracy: 0.4505 - val_loss: 1.5905
Epoch 7/15
28/28 ————— 157s 6s/step - accuracy: 0.4494 - loss: 1.5400 - val_accuracy: 0.4414 - val_loss: 1.5694
Epoch 8/15
28/28 ————— 157s 6s/step - accuracy: 0.4494 - loss: 1.5313 - val_accuracy: 0.4369 - val_loss: 1.5475
Epoch 9/15
28/28 ————— 157s 6s/step - accuracy: 0.4494 - loss: 1.5227 - val_accuracy: 0.4369 - val_loss: 1.5357
Epoch 10/15
28/28 ————— 157s 6s/step - accuracy: 0.4448 - loss: 1.5197 - val_accuracy: 0.4459 - val_loss: 1.5295
Epoch 11/15
28/28 ————— 157s 6s/step - accuracy: 0.4482 - loss: 1.5092 - val_accuracy: 0.4459 - val_loss: 1.5163
Epoch 12/15
28/28 ————— 157s 6s/step - accuracy: 0.4482 - loss: 1.5024 - val_accuracy: 0.4459 - val_loss: 1.5042
Epoch 13/15
28/28 ————— 157s 6s/step - accuracy: 0.4482 - loss: 1.4955 - val_accuracy: 0.4459 - val_loss: 1.4976
Epoch 14/15
28/28 ————— 156s 6s/step - accuracy: 0.4482 - loss: 1.4895 - val_accuracy: 0.4459 - val_loss: 1.4917
Epoch 15/15
28/28 ————— 158s 6s/step - accuracy: 0.4482 - loss: 1.4853 - val_accuracy: 0.4459 - val_loss: 1.4863
```

```
# need this because colab reset
train_dir = "/content/alzheimers_working/train"
val_dir = "/content/alzheimers_working/val"
test_dir = "/content/alzheimers_working/test"
```

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
aug_datagen = ImageDataGenerator(
    rescale=1./255,
    samplewise_center=True,
    samplewise_std_normalization=True,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True
)
```

```
val_aug_datagen = ImageDataGenerator(
    rescale=1./255,
    samplewise_center=True,
    samplewise_std_normalization=True
)
```

```
train_aug = aug_datagen.flow_from_directory(
    train_dir,
```

```
target_size=IMG_SIZE,
batch_size=BATCH_SIZE,
class_mode='categorical',
shuffle=True
)

val_aug = val_aug_datagen.flow_from_directory(
    val_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)
```

Found 879 images belonging to 5 classes.
Found 222 images belonging to 5 classes.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization

cnn_model_1_aug = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224, 224, 3)),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax')
])

cnn_model_1_aug.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

cnn_model_1_aug.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 222, 222, 32)	896
batch_normalization_9 (BatchNormalization)	(None, 222, 222, 32)	128
max_pooling2d_9 (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_10 (Conv2D)	(None, 109, 109, 64)	18,496
batch_normalization_10 (BatchNormalization)	(None, 109, 109, 64)	256
max_pooling2d_10 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_11 (Conv2D)	(None, 52, 52, 128)	73,856
batch_normalization_11 (BatchNormalization)	(None, 52, 52, 128)	512
max_pooling2d_11 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten_3 (Flatten)	(None, 86528)	0
dense_8 (Dense)	(None, 128)	11,075,712
dropout_4 (Dropout)	(None, 128)	0
dense_9 (Dense)	(None, 5)	645

Total params: 11,170,501 (42.61 MB)

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

```
checkpoint = ModelCheckpoint(
    "/content/drive/MyDrive/best_cnn_model_1_aug.keras",
    monitor='val_loss',
    save_best_only=True
)
```

```
history_1_aug = cnn_model_1_aug.fit(
    train_aug,
    validation_data=val_aug,
    epochs=15,
    callbacks=[early_stop, checkpoint]
)
```

Epoch 1/15
28/28  172s 6s/step - accuracy: 0.3697 - loss: 6.9063 - val_accuracy: 0.4459 - val_loss: 1.6708
Epoch 2/15
28/28  204s 6s/step - accuracy: 0.4482 - loss: 1.6111 - val_accuracy: 0.4459 - val_loss: 1.6318
Epoch 3/15
28/28  172s 6s/step - accuracy: 0.4448 - loss: 1.5822 - val_accuracy: 0.4459 - val_loss: 1.6236
Epoch 4/15
28/28  172s 6s/step - accuracy: 0.4482 - loss: 1.5656 - val_accuracy: 0.4459 - val_loss: 1.6608
Epoch 5/15
28/28  172s 6s/step - accuracy: 0.4482 - loss: 1.5586 - val_accuracy: 0.3514 - val_loss: 1.8199
Epoch 6/15
28/28  166s 6s/step - accuracy: 0.4471 - loss: 1.5822 - val_accuracy: 0.4459 - val_loss: 1.8963

```
transfer_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_transfer = transfer_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    shuffle=True
)
```

```
val_transfer = transfer_datagen.flow_from_directory(
    val_dir
```

```
val_size,
target_size=(224, 224),
batch_size=32,
class_mode='categorical',
shuffle=False
)
```

Found 879 images belonging to 5 classes.
Found 222 images belonging to 5 classes.

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False

transfer_model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(5, activation='softmax')
])

transfer_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

transfer_model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_order0_0_224/9406464/9406464 1s 0us/step
Model: "sequential_1"

Layer (type)	Output Shape	Param #
mobilenetv2_1_00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 5)	645

Total params: 2,422,597 (9.24 MB)
Trainable params: 164,613 (643.02 KB)


```

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

checkpoint = ModelCheckpoint(
    "/content/drive/MyDrive/best_transfer_model.keras",
    monitor='val_loss',
    save_best_only=True
)

history_transfer = transfer_model.fit(
    train_transfer,
    validation_data=val_transfer,
    epochs=15,
    callbacks=[early_stop, checkpoint]
)

```

```

Epoch 1/15
28/28 ————— 53s 2s/step - accuracy: 0.3788 - loss: 1.6181 - val_accuracy: 0.4324 - val_loss: 1.4227
Epoch 2/15
28/28 ————— 45s 2s/step - accuracy: 0.4152 - loss: 1.4315 - val_accuracy: 0.4595 - val_loss: 1.3696
Epoch 3/15
28/28 ————— 44s 2s/step - accuracy: 0.4334 - loss: 1.3933 - val_accuracy: 0.4595 - val_loss: 1.3651
Epoch 4/15
28/28 ————— 43s 2s/step - accuracy: 0.4585 - loss: 1.3764 - val_accuracy: 0.4505 - val_loss: 1.3885
Epoch 5/15
28/28 ————— 44s 2s/step - accuracy: 0.4516 - loss: 1.3554 - val_accuracy: 0.4730 - val_loss: 1.3551
Epoch 6/15
28/28 ————— 48s 2s/step - accuracy: 0.4687 - loss: 1.2893 - val_accuracy: 0.4505 - val_loss: 1.3493
Epoch 7/15
28/28 ————— 79s 2s/step - accuracy: 0.4812 - loss: 1.2755 - val_accuracy: 0.4595 - val_loss: 1.3441
Epoch 8/15
28/28 ————— 45s 2s/step - accuracy: 0.4869 - loss: 1.2547 - val_accuracy: 0.4685 - val_loss: 1.3372
Epoch 9/15
28/28 ————— 44s 2s/step - accuracy: 0.4972 - loss: 1.2475 - val_accuracy: 0.4414 - val_loss: 1.3686
Epoch 10/15
28/28 ————— 45s 2s/step - accuracy: 0.4733 - loss: 1.2412 - val_accuracy: 0.4640 - val_loss: 1.3386
Epoch 11/15
28/28 ————— 45s 2s/step - accuracy: 0.5085 - loss: 1.2203 - val_accuracy: 0.4550 - val_loss: 1.3301
Epoch 12/15
28/28 ————— 45s 2s/step - accuracy: 0.5028 - loss: 1.2045 - val_accuracy: 0.4595 - val_loss: 1.3174
Epoch 13/15
28/28 ————— 44s 2s/step - accuracy: 0.5233 - loss: 1.1595 - val_accuracy: 0.4595 - val_loss: 1.3255
Epoch 14/15
28/28 ————— 80s 2s/step - accuracy: 0.5313 - loss: 1.1461 - val_accuracy: 0.4820 - val_loss: 1.3235
Epoch 15/15
28/28 ————— 44s 2s/step - accuracy: 0.5552 - loss: 1.1173 - val_accuracy: 0.4685 - val_loss: 1.3444

```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
test_datagen = ImageDataGenerator(rescale=1./255)
```

```

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)

```

Found 195 images belonging to 5 classes.

```
from tensorflow.keras.models import load_model
```

```

best_cnn_model = load_model("/content/drive/MyDrive/best_cnn_model_1.keras")
best_transfer_model = load_model("/content/drive/MyDrive/best_transfer_model.keras")

```

```

cnn_test_loss, cnn_test_acc = best_cnn_model.evaluate(test_generator)
print("CNN Test Accuracy:", cnn_test_acc)
print("CNN Test Loss:", cnn_test_loss)

```

```

transfer_test_loss, transfer_test_acc = best_transfer_model.evaluate(test_generator)
print("Transfer Model Test Accuracy:", transfer_test_acc)
print("Transfer Model Test Loss:", transfer_test_loss)

```

```

7/7  9s 1s/step - accuracy: 0.4462 - loss: 8.5765
CNN Test Accuracy: 0.446153849363327
CNN Test Loss: 8.57646656036377
7/7  10s 1s/step - accuracy: 0.4513 - loss: 1.3406
Transfer Model Test Accuracy: 0.4512820541858673

```

```

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

# reset generator before predictions
test_generator.reset()

cnn_preds = best_cnn_model.predict(test_generator)
cnn_pred_classes = np.argmax(cnn_preds, axis=1)

true_classes = test_generator.classes
class_labels = list(test_generator.class_indices.keys())

print("CNN Classification Report")
print(classification_report(true_classes, cnn_pred_classes, target_names=class_labels))

print("CNN Confusion Matrix")
print(confusion_matrix(true_classes, cnn_pred_classes))

```

```

7/7  11s 1s/step
CNN Classification Report

```

	precision	recall	f1-score	support
Final AD JPEG	0.00	0.00	0.00	26
Final CN JPEG	0.45	1.00	0.62	87
Final EMCI JPEG	0.00	0.00	0.00	36
Final LMCI JPEG	0.00	0.00	0.00	11
Final MCI JPEG	0.00	0.00	0.00	35
accuracy			0.45	195
macro avg	0.09	0.20	0.12	195
weighted avg	0.20	0.45	0.28	195

CNN Confusion Matrix

```

[[ 0 26  0  0  0]
 [ 0 87  0  0  0]
 [ 0 36  0  0  0]
 [ 0 11  0  0  0]
 [ 0 35  0  0  0]]

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

```

test_generator.reset()

transfer_preds = best_transfer_model.predict(test_generator)
transfer_pred_classes = np.argmax(transfer_preds, axis=1)

print("Transfer Learning Classification Report")
print(classification_report(true_classes, transfer_pred_classes, target_names=class_labels))

```